

The Polar Express: Optimal Matrix Sign Methods and Their Application to the Muon Algorithm

Noah Amsel* David Persson† Christopher Musco‡ Robert M. Gower§

September 29, 2025

This paper is part of the project that me and my group are working on

The question I asked here is ‘What is the matrix sign’

Abstract

Computing the polar decomposition and the related matrix sign function has been a well-studied problem in numerical analysis for decades. Recently, it has emerged as an important subroutine within the Muon algorithm for training deep neural networks. However, the requirements of this application differ sharply from classical settings: deep learning demands GPU-friendly algorithms that prioritize high throughput over high precision. We introduce **Polar Express**, a new method for computing the polar decomposition.¹ Like Newton–Schulz and other classical polynomial methods, our approach uses only matrix-matrix multiplications, making it very efficient on GPUs. Inspired by earlier work of Chen & Chow and Nakatsukasa & Freund, **Polar Express** adapts the update rule at each iteration by solving a minimax optimization problem. We prove that this strategy minimizes error in a worst-case sense, allowing **Polar Express** to converge as rapidly as possible both in the early iterations and asymptotically. We also address finite-precision issues, making it practical to use in `bf16`. When integrated into the Muon training framework, our method leads to consistent improvements in validation loss when training a GPT-2 model on one billion tokens from the FineWeb dataset, outperforming recent alternatives across a range of learning rates.

1 Introduction

Advanced linear algebra is making its way into deep learning. Efficient algorithms for computing *matrix functions* have found exciting new applications in training neural networks. In particular, approximations to the matrix-inverse are used in the full Adagrad method [15], the matrix square-root and quarter-root appear as subroutines in the Shampoo and Soap optimizers [20, 48, 51], and most recently, the matrix sign function has become a key ingredient of the Muon optimizer [5, 4, 25].

While the problem of computing these matrix functions has been studied by numerical analysts for decades, applications in deep learning come with different requirements than those in computational science. For deep learning, it is critical to take maximum advantage of GPU-friendly operations like matrix-matrix products and to avoid less parallel operations. Moreover, memory overhead must be small to handle large models. On the other hand,

*New York University. noah.amsel@nyu.edu

†New York University and Flatiron Institute. dup210@nyu.edu, dperisson@flatironinstitute.org

‡New York University. cmusco@nyu.edu

§Flatiron Institute. rgower@flatironinstitute.org

¹<https://github.com/NoahAmsel/PolarExpress>

Here the LLM is incorrect. Muon does not compute the matrix sign. It computes the polar decomposition. We will see how these are different in one of the following annotations

1

The **matrix sign function** is a matrix-valued analogue of the scalar sign function $\text{sign}(x)$.

For a diagonalizable matrix A with no eigenvalues on the imaginary axis, you define $\text{sign}(A)$ by applying sign to its eigenvalues:

- If $A = V\Lambda V^{-1}$ with eigenvalues λ_i , then

$$\text{sign}(A) = V \text{sign}(\Lambda) V^{-1},$$

where $\text{sign}(\Lambda)$ is diagonal with entries

$$\text{sign}(\lambda_i) = \begin{cases} +1 & \text{if } \Re(\lambda_i) > 0, \\ -1 & \text{if } \Re(\lambda_i) < 0. \end{cases}$$

For a real square matrix A with polar decomposition $A = QH$ (where Q is orthogonal/unitary and H is positive semidefinite), the matrix sign is closely related:

$$\text{sign}(A) = Q \quad \text{when } A \text{ is nonsingular and } H \succ 0.$$

So, in many applications (including Muon), computing the matrix sign is effectively a way to extract the “orthogonal factor” Q from a matrix, using only matrix-matrix operations.

I asked the LLM to explain the difference between You and Jordan method

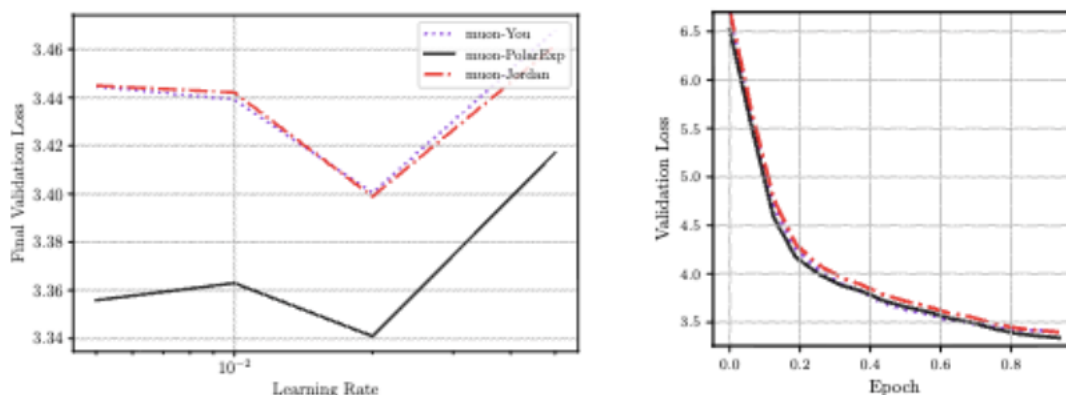


Figure 1: Training a GPT-2-Large model (774M params) on 1 billion tokens from the FineWeb dataset [2]. The label muon-`<name>` refers to implementing Muon using `<name>` to compute the polar factor. Left: final validation loss across learning rates. Right: validation loss across epochs using the best learning rate. The best learning rate (lr) and final validation loss for each method was adamw ($lr = 0.0001$): 4.172, muon-You ($lr = 0.02$): 3.400, muon-Jordan ($lr = 0.02$): 3.398 and muon-PolarExp ($lr = 0.02$): 3.340.

1.1 The Muon Method

The Muon optimizer has recently gained popularity for training large language models, often outperforming state-of-the-art adaptive gradient methods like Adam and AdamW [30, 35]. Muon has been used to set records for the NanoGPT speedrun [25], to expand the Pareto frontier of performance versus training FLOPs for large language models [34, 47], and even to train a 32-billion parameter frontier LLM [29].

The Muon update rule [5] is defined as follows. Let $\lambda, \beta > 0$ be the learning rate and momentum coefficient hyperparameters. (By default, $\beta = 0.9$.) Let $\mathbf{W}_t \in \mathbb{R}^{m \times n}$ be the weight matrix of a given neural network layer at iteration t , and let $\mathbf{G}_t \in \mathbb{R}^{m \times n}$ be its (stochastic) gradient. Let $\mathbf{M}_t \in \mathbb{R}^{m \times n}$ be the running momentum estimate of the gradient, where $\mathbf{M}_0 = \mathbf{0}$. The Muon update is given by

$$\begin{aligned} \mathbf{M}_t &= \beta \mathbf{M}_{t-1} + (1 - \beta) \mathbf{G}_t \\ \mathbf{W}_{t+1} &= \mathbf{W}_t - \lambda \text{polar}(\mathbf{M}_t). \end{aligned}$$

Whereas standard stochastic gradient descent (SGD) with momentum updates the weight matrix by taking a step in the direction $-\mathbf{M}_t$, the Muon method steps in the direction $-\text{polar}(\mathbf{M}_t)$, where $\text{polar}(\mathbf{M})$ denotes the closest semi-orthogonal matrix to $\mathbf{M}$ [22, Chapter 8]. Concretely, if $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ is the singular value decomposition (SVD) of $\mathbf{M}$, then

$$\text{polar}(\mathbf{M}) := \mathbf{U}\mathbf{V}^\top. \quad (1)$$

The matrix $\text{polar}(\mathbf{M})$ can be seen as a generalization of the matrix sign function to rectangular matrices [3]. Indeed, when $\mathbf{M}$ is square symmetric with eigendecomposition $\mathbf{M} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$, $\text{polar}(\mathbf{M})$ exactly coincides with the matrix sign function $\text{sign}(\mathbf{M}) = \mathbf{V} \text{sign}(\mathbf{\Lambda}) \mathbf{V}^\top$ [22, Chapter 5]. Equivalently, $\text{polar}(\mathbf{M})$ is the left orthogonal factor of the polar decomposition of $\mathbf{M}$ [22, Chapter 8]. The motivation for Muon is that $-\text{polar}(\mathbf{M})$ gives the steepest-descent direction with respect to the *spectral norm* (instead of the Frobenius norm, as in standard SGD).

The LLM correctly interprets the figure caption to denote the results of the graph for You and Jordan methods.

Here we come back to the original, correct claim: Muon approximates the polar factor via iterated polynomial composition

4

Muon-You vs. Muon-Jordan:

- Both are the same optimizer (Muon); they differ only in **how they approximate the polar factor** ``polar(M_t)`` in the update

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \lambda \text{polar}(\mathbf{M}_t).$$

It would have been more helpful if the LLM provided the actual algorithm instead of saying 'You et al.'s iterative algorithm' or 'classical Jordan iteration' - **Muon-You**: uses You et al.'s iterative

method for the matrix sign/polar decomposition (a rational iteration similar to Newton-Schulz).

- **Muon-Jordan**: uses the classical Jordan (or "scaled Newton") iteration for the polar decomposition.

So the difference is purely in the **numerical algorithm** to compute ``polar(M_t)``—not in the high-level optimization rule.

Which is better?

From the figure you attached and the numbers in the text:

2

- Muon-You (best lr 0.02): final validation loss $\approx$ **3.400**

- Muon-Jordan (best lr 0.02): final validation loss $\approx$ **3.398**

They perform **essentially the same**, with a tiny edge to Muon-Jordan in this experiment (difference well within typical run-to-run variance). The more meaningful comparison is that both are much better than AdamW here, and both are slightly worse than Muon-PolarExp (3.340).

[30, 35]. Muon has been used to set records for the NanoGPT speedrun [25], to expand the Pareto frontier of performance versus training FLOPs for large language models [34, 47], and even to train a 32-billion parameter frontier LLM [29].

The Muon update rule [5] is defined as follows. Let $\lambda, \beta > 0$ be the learning rate and momentum coefficient hyperparameters. (By default, $\beta = 0.9$.) Let $\mathbf{W}_t \in \mathbb{R}^{m \times n}$ be the weight matrix of a given neural network layer at iteration t , and let $\mathbf{G}_t \in \mathbb{R}^{m \times n}$ be its (stochastic) gradient. Let $\mathbf{M}_t \in \mathbb{R}^{m \times n}$ be the running momentum estimate of the gradient, where $\mathbf{M}_0 = \mathbf{0}$. The Muon update is given by

$$\begin{aligned}\mathbf{M}_t &= \beta \mathbf{M}_{t-1} + (1 - \beta) \mathbf{G}_t \\ \mathbf{W}_{t+1} &= \mathbf{W}_t - \lambda \text{polar}(\mathbf{M}_t).\end{aligned}$$

Whereas standard stochastic gradient descent (SGD) with momentum updates the weight matrix by taking a step in the direction $-\mathbf{M}_t$, the Muon method steps in the direction $-\text{polar}(\mathbf{M}_t)$, where $\text{polar}(\mathbf{M})$ denotes the closest semi-orthogonal matrix to $\mathbf{M}$ [22, Chapter 8]. Concretely, if $\mathbf{M} = \mathbf{U}\Sigma\mathbf{V}^\top$ is the singular value decomposition (SVD) of $\mathbf{M}$, then

$$\text{polar}(\mathbf{M}) := \mathbf{U}\mathbf{V}^\top. \quad (1)$$

The matrix $\text{polar}(\mathbf{M})$ can be seen as a generalization of the matrix sign function to rectangular matrices [3]. Indeed, when $\mathbf{M}$ is square symmetric with eigendecomposition $\mathbf{M} = \mathbf{V}\Lambda\mathbf{V}^\top$, $\text{polar}(\mathbf{M})$ exactly coincides with the matrix sign function $\text{sign}(\mathbf{M}) = \mathbf{V} \text{sign}(\Lambda) \mathbf{V}^\top$ [22, Chapter 5]. Equivalently, $\text{polar}(\mathbf{M})$ is the left orthogonal factor of the polar decomposition of $\mathbf{M}$ [22, Chapter 8]. The motivation for Muon is that $-\text{polar}(\mathbf{M})$ gives the steepest-descent direction with respect to the *spectral norm* (instead of the Frobenius norm, as in standard SGD).

3

As we can see, these two Matrix functions are different. And the $\mathbf{U}\mathbf{V}^\top$ from one of our previous lectures shows up. This is what Muon is trying to approximate.

The question I asked here was ‘What is the difference between Polar (M) and Matrix Sign (M)’

2

Not in general.

- For any (possibly rectangular) matrix $\mathbf{M}$ with SVD $\mathbf{M} = \mathbf{U}\Sigma\mathbf{V}^\top$, the **polar factor** is

$$\text{polar}(\mathbf{M}) = \mathbf{U}\mathbf{V}^\top,$$

which is semi-orthogonal (orthonormal columns/rows) and is defined for rectangular matrices.

- The **matrix sign** is defined only for **square** matrices that are diagonalizable with no eigenvalues at 0. If $\mathbf{M}$ is **symmetric** with eigendecomposition $\mathbf{M} = \mathbf{V}\Lambda\mathbf{V}^\top$, then

$$\text{sign}(\mathbf{M}) = \mathbf{V} \text{sign}(\Lambda) \mathbf{V}^\top,$$

with eigenvalues ± 1 .

The paper notes: when $\mathbf{M}$ is **square symmetric**, $\text{polar}(\mathbf{M})$ *coincides* with $\text{sign}(\mathbf{M})$. Outside that special case (e.g., rectangular or non-symmetric $\mathbf{M}$), they differ, and only $\text{polar}(\mathbf{M})$ is well-defined in the way they use it.

Recent work [45] shows that Muon can be viewed as a conditional gradient (Frank-Wolfe) method with a trust region defined by the spectral norm. In the same work, the authors also provide a convergence theory for the smooth and non-convex setting, as well as for the stochastic non-convex case. The analysis of Muon was further refined in [46], which proves convergence under a layerwise (L_0, L_1) -smoothness assumption, in both the stochastic non-convex and stochastic Polyak–Łojasiewicz settings. We also note earlier work that anticipated Muon’s use of the polar factor and its motivation as steepest descent under the spectral norm [8, 9]. We refer the reader to [25] and [5] for further background. In this paper, we take the Muon update rule as given and focus on the problem of efficiently computing the polar decomposition $\text{polar}(\mathbf{M})$.

1.2 Computing the Polar Factor

Although $\text{polar}(\mathbf{M})$ can be computed directly via an SVD in $O(\min(mn^2, nm^2))$ time, doing so is prohibitively expensive, especially as standard SVD algorithms fail to take full advantage of GPU acceleration. There has been significant work on highly-parallel approaches, but the most common approaches actually require computing the matrix-sign function as a subroutine [38, 40]. Numerical analysts have spent decades developing iterative methods for computing $\text{polar}(\mathbf{M})$. This

The Newton-Schulz methods. Newton-Schulz constructs a sequence of approximations $X_t \approx \text{polar}(M)$ as follows:

$$X_0 = M / \|M\|_F \quad X_{t+1} = \frac{3}{2}X_t - \frac{1}{2}X_t X_t^\top X_t \quad (2)$$

At each iteration, this rule effectively applies the cubic polynomial $p(x) = \frac{3}{2}x - \frac{1}{2}x^3$ to each singular value of X_t . The scalar fixed-point iteration $x_{t+1} = p(x_t)$ converges to $\text{sign}(x_0)$ as $t \rightarrow \infty$, provided $|x_0| \leq 1$. As a result, the matrix iteration satisfies $\lim_{t \rightarrow \infty} X_t = UV^\top = \text{polar}(X_0)$. Higher-degree versions of Newton-Schulz follow the same principle. For example, the degree-5 polynomial $p(x) = (15x - 10x^3 + 3x^5)/8$ converges even faster. The Newton-Schulz iterations converge super-exponentially when X_t is sufficiently close to $\text{polar}(M)$, but they suffer from slow initial convergence; when X_0 is far from $\text{polar}(M)$, the approximation improves slowly over the first few iterations.

The Jordan and You methods. In Muon, high accuracy approximations to $\text{polar}(M)$ are usually not necessary. The primary goal is instead to compute a coarse approximation in as few iterations as possible. To accelerate convergence in the low-accuracy regime, Jordan recently proposed a fixed-point iteration based on the polynomial $p(x) = 3.4445x -$

While it gave the correct polynomial and explanation⁴, it would have been more clear if it did the decomposition of the SVD and showed that a polynomial operating on a matrix only acts on the sigma matrix (like how we learned in class), though this might have been an assumption the authors made about the audience's background knowledge

$4.7750x^3 + 2.0315x^5$ [25], which was found using a heuristic numerical search. Unlike Newton-Schulz, the scheme that Jordan proposed does not converge to $\text{polar}(M)$. Instead, it plateaus at an error of ≈ 0.3 . However, it reaches this level of accuracy rapidly. As a result, when the number of iterations is smaller than about 10, Jordan's method outperforms the Newton-Schulz iteration. Building on this idea, You [10] proposed a method that applies six different polynomial updates in succession, which were again found by heuristic search. This method achieves better accuracy than Jordan's but still fails to converge.

We introduce a new method. In particular, we derive polynomial update rules that are optimal at every iteration, outperforming all previous polynomial methods in our setting.

1.3 Contributions

We present **Polar Express**², an iterative method for approximating $\text{polar}(M)$. Our method dynamically adapts the polynomial update rule at each iteration, prioritizing rapid progress in the initial stage and high accuracy in the later stage. **Polar Express** constructs polynomials $p_1, \dots, p_T$ so that the resulting composition is the optimal approximation to the sign function with respect to the supremum (L^∞) norm (Theorem 4.1). By iteratively applying these polynomials to M , **Polar Express** computes an approximation to $\text{polar}(M)$ that is optimal in the worst-case at every iteration. Our method converges to $\text{polar}(M)$ super-exponentially (Theorem 4.3), and it quickly reaches a good approximation within just five to ten iterations. This early-stage acceleration is especially valuable in deep learning applications, where runtime efficiency takes precedence over high accuracy. In contrast, classical methods like Newton-Schulz suffer from a slow initial convergence, while recent heuristic proposals [25, 10] fail to converge. Our method is efficient to run on GPUs, using only a few matrix-matrix products per iteration.³

We give an explicit instantiation of **Polar Express** in Algorithm 1, which incorporates minor modifications to make it compatible with half-precision arithmetic (see Section 4.4). Algorithm 1 can be used as a drop-in replacement for previous methods. In numerical experiments, **Polar Express** outperforms previous methods on synthetic matrices and gradient matrices from a GPT-2 transformer (Figure 4). We demonstrate the effectiveness of using **Polar Express** within the Muon optimizer in Figure 1, showing that it consistently improves the training of GPT-2 language models on 1 billion tokens of the FineWeb dataset [2].

Notation. We let $\|M\|_F$ denote the Frobenius norm and spectral norm (largest singular value) of M . We let $\sigma(M)$ denote the spectrum (set of singular values) by $\sigma(M)$.

Let $\mathbb{P}_d$ be the set of polynomials of degree at most d . For odd d , $\mathbb{P}_d^{\text{odd}}$ denotes the set of polynomials of degree at most d containing only odd-degree monomials. For a polynomial p , $\deg(p)$ is its degree. Let $\text{sign}(x)$ be the scalar sign function, which satisfies $\text{sign}(0) = 0$.

Here I asked to explain what Newton Schulz is

3

Newton-Schulz is an **iterative algorithm to compute the polar factor** (or matrix sign) using only matrix multiplications, no inverses/QR. That's why it's attractive for GPUs.

For a matrix M with polar decomposition $M = UP$ (where U is orthogonal and P is positive semidefinite), Newton-Schulz constructs matrices X_t that converge to $U = \text{polar}(M)$:

1. Initialize

$$X_0 = \frac{M}{\|M\|_F}.$$

2. Iterate

$$X_{t+1} = \frac{3}{2}X_t - \frac{1}{2}X_t X_t^\top X_t.$$

Here the LLM is incorrect. Muon does not compute the matrix sign. It computes the polar decomposition. We will see how these are different in one of the following annotations

To understand why this works, look at a single singular value σ of X_t . The iteration applies the scalar cubic polynomial

$$p(x) = \frac{3}{2}x - \frac{1}{2}x^3$$

to each singular value. The scalar map

$$x_{t+1} = p(x_t)$$

converges to $\text{sign}(x_0)$ for $|x_0| \leq 1$. On the matrix level this means the singular values are driven toward 1 (for nonzero ones), so X_t becomes more and more orthogonal and

$$X_t \rightarrow U = \text{polar}(M).$$

This is true, Newton Schulz does have a very slow initial convergence

Key properties:

- Uses only matrix-matrix multiplies (GPU-friendly).
- Converges **super-exponentially** when already close to $\text{polar}(M)$.
- Has **slow initial convergence** when X_0 is far from the polar factor.

Hence, for an odd polynomial $p(x) = a_0x + a_1x^3 + \dots + a_qx^{2q+1}$ we can compute

$$p(\mathbf{M}) := a_0\mathbf{M} + a_1\mathbf{M}(\mathbf{M}^\top\mathbf{M}) + \dots + a_q\mathbf{M}(\mathbf{M}^\top\mathbf{M})^q.$$

It has been shown that for an arbitrary polynomial p , one requires $\Theta(\deg(p)^{1/2})$ products to compute $p(\mathbf{M})$ [44]; see also [23] for related work. This compares favorably to the naive approach that forms all monomials in p and then sums them together, which requires $\Omega(\deg(p))$ products. However, if p can be expressed as a composition of T polynomials, each of degree d

$$p = p_T \circ p_{T-1} \circ \dots \circ p_1, \quad (4)$$

then the degree of p is d^T , and $p(\mathbf{M})$ can be efficiently computed recursively by

$$\mathbf{X}_0 = \mathbf{M}, \quad \mathbf{X}_t = p_t(\mathbf{X}_{t-1}) \text{ for } t = 1, 2, \dots, T. \quad (5)$$

The final iterate is $\mathbf{X}_T = p(\mathbf{M})$, which we compute with just $O(Td)$ matrix-matrix products.

Iterative methods for $\text{polar}(\mathbf{M})$ can be seen in this light. For instance, the degree-5 Newton-Schulz method uses the polynomial update $p_t(x) = \frac{15}{8}x - \frac{10}{8}x^3 + \frac{3}{8}x^5$ for each $t = 1, \dots, T$. The composition $p = p_T \circ \dots \circ p_1$ approximates $\text{sign}(x)$, and the approximation error goes to 0 as T grows. In this paper, we ask the following question: what choice of $p_T \circ \dots \circ p_1$ gives the *best* approximation to $\text{sign}(x)$?

The method we will present is optimal in the following sense: given lower and upper bounds ℓ and u on the singular values of $\mathbf{M}$, an odd degree $d \in \mathbb{N}$, and the number of iterations $T \in \mathbb{N}$, our method computes the composition $p^*(\mathbf{M})$ that minimizes the worst-case error in the spectral norm. That is,

$$p^* = \underset{\substack{p = p_T \circ p_{T-1} \circ \dots \circ p_1 \\ p_t \in \mathbb{P}_d^{\text{odd}}}}{\text{argmin}} \max_{\substack{\mathbf{M} \in \mathbb{R}^{m \times n} \\ \sigma(\mathbf{M}) \subset [\ell, u]}} \|\text{polar}(\mathbf{M}) - p(\mathbf{M})\|_2. \quad (6)$$

Given that $\text{polar}(\mathbf{M}) - p(\mathbf{M}) = \mathbf{U}(\mathbf{I} - p(\boldsymbol{\Sigma}))\mathbf{V}^\top$, and by the unitary invariance of the spectral norm, we have that (6) is equivalent to

$$p^* = \underset{\substack{p = p_T \circ p_{T-1} \circ \dots \circ p_1 \\ p_t \in \mathbb{P}_d^{\text{odd}}}}{\text{argmin}} \max_{x \in [\ell, u]} |1 - p(x)|. \quad (7)$$

For completeness, the equivalence between (6) and (7) is proven in [Appendix C](#).

In other words, the problem given in (6) reduces to that of finding a “uniform” or “minimax” approximation to the constant function $x \mapsto 1$ over the interval $[\ell, u]$, as given in (7). Uniform approximation on an interval by polynomials or rational functions of a given degree is a central topic in approximation theory; see e.g. [50]. Here, we seek an approximation of a particular form—a *composition* of odd polynomials of fixed degrees. In the next section, we solve the optimization problem of (7) and use the solution to create **Polar Express**. [Figure 2](#) (a) shows the resulting p^* polynomial labeled as **PolarExp**, as compared to the **Jordan’s** method in [25], and the six iterations of **You’s** method in [10].

Here I asked why odd polynomial

Two reasons:

1. **Target function is odd (matrix sign / polar factor):**

For an SVD $\mathbf{M} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$, the polar factor is $\text{polar}(\mathbf{M}) = \mathbf{U}\mathbf{V}^\top = \mathbf{M}(\mathbf{M}^\top\mathbf{M})^{-1/2}$. This $\mathbf{M}(\mathbf{M}^\top\mathbf{M})^{-1/2}$ is an interesting way of defining the polar factor since it does simplify to $\mathbf{U}\mathbf{V}^\top$, but we never

$$\text{polar}(\mathbf{M}) = \mathbf{U}\mathbf{V}^\top = \mathbf{M}(\mathbf{M}^\top\mathbf{M})^{-1/2}.$$

As a scalar function of a singular value $x > 0$, this looks like $x \mapsto 1$ *multiplied by the direction of $\mathbf{M}^*$. To preserve the left/right singular vectors and only rescale singular values, the update must be of the form

$$p(\mathbf{M}) = \mathbf{M} q(\mathbf{M}^\top\mathbf{M}),$$

which corresponds exactly to an **odd** scalar polynomial $p(x) = x q(x^2)$. Even terms $(x^0, x^2, x^4, \dots)$ would produce components that are not of the form $\mathbf{M}$ times something symmetric, and wouldn’t preserve the structure needed for the polar factor.

2. ****Efficient computation using $\mathbf{M}^\top\mathbf{M}$ only:****

For odd powers,

$$\mathbf{M}^{2q+1} = \mathbf{M}(\mathbf{M}^\top\mathbf{M})^q,$$

so any odd polynomial

$$p(x) = a_0x + a_1x^3 + \dots + a_qx^{2q+1}$$

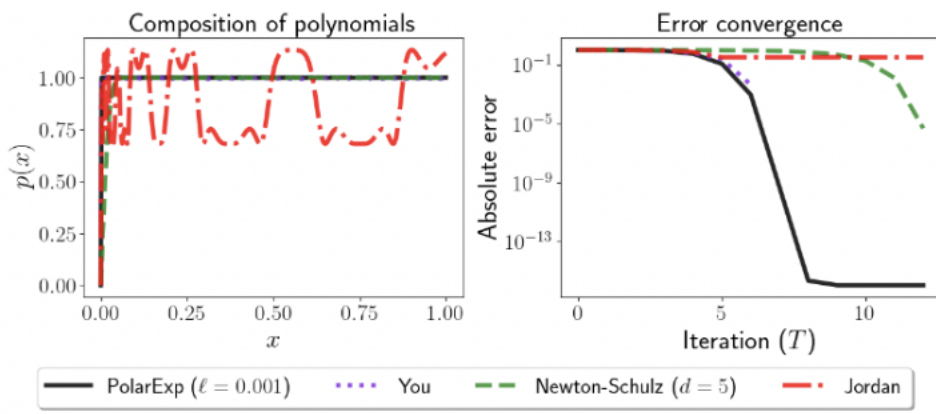
gives

$$p(\mathbf{M}) = a_0\mathbf{M} + a_1\mathbf{M}(\mathbf{M}^\top\mathbf{M}) + \dots + a_q\mathbf{M}(\mathbf{M}^\top\mathbf{M})^q,$$

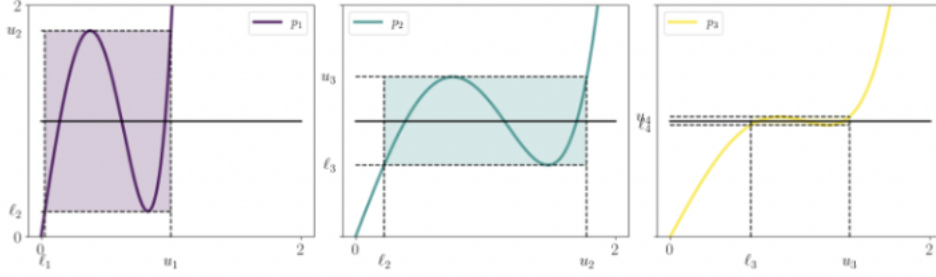
which can be built from repeated products with $\mathbf{M}^\top\mathbf{M}$. This is cheap and stable. There is no analogous simple expression for even powers $\mathbf{M}^{2q}$ in terms of $\mathbf{M}^\top\mathbf{M}$ without extra SVD-like work.

So odd polynomials are both:

- structurally aligned with $\text{polar}(\mathbf{M})$, and
- computationally convenient to evaluate via $\mathbf{M}^\top\mathbf{M}$.



(a) The left figure compares the composition (for $T = 6$ and $d = 5$) of polynomials given by Polar Express ($\ell = 0.001$), You's method (which is defined up to 6 iterations), Newton-Schulz, and Jordan's method for approximating $\text{sign}(x)$. The right figure demonstrates the convergence of the methods on $[0.001, 1]$. Note the slow initial convergence of Newton-Schulz.



(b) The evolution of the first three optimal polynomials p_1 , p_2 , and p_3 and the corresponding lower bounds $\ell_{t+1} = p_t(\ell_t)$ and upper bounds $u_{t+1} = 2 - \ell_{t+1}$, as described in Theorem 4.1. The horizontal black line indicates $y = 1$. The polynomial degree is $d = 5$ and the number of iterations is $T = 3$. We set $\ell_1 = 0.03$ and $u_1 = 1$.

Figure 2

4 The Polar Express

4.1 Greedy is optimal

The key observation is that the polynomial used in each iteration can be chosen greedily, given the choice of polynomials from the previous iterations. For the first iteration, we choose p_1 so as to map the interval $[\ell, u]$ as close to 1 as possible. That is, it minimizes $\max_{x \in [\ell, u]} |1 - p_1(x)|$. The image of p_1 will be a new interval $[\ell_2, u_2]$, where

$$\ell_2 = \min_{x \in [\ell, u]} p_1(x) \quad u_2 = \max_{x \in [\ell, u]} p_1(x) \quad (8)$$

We now pick p_2 to map the interval $[\ell_2, u_2]$ as close to 1 as possible, obtaining a new interval $[\ell_3, u_3]$ that is the image of $[\ell, u]$ through $p_2 \circ p_1$. We continue this process for as many iterations as desired.

10

The following theorem guarantees that this process finds the solution to (7), and thereby also (6). The scheme is also outlined in Figure 2 (b), which demonstrates the evolution of the lower bounds ℓ_t , the upper bounds u_t , and the polynomials p_t across iterations.

Theorem 4.1. Let d be odd and define $\ell_1 = \ell$ and $u_1 = u$. For $t = 1, \dots, T$ define

$$\begin{aligned} p_t &= \operatorname{argmin}_{p \in \mathbb{P}_d^{\text{odd}}} \max_{x \in [\ell_t, u_t]} |1 - p(x)| \\ \ell_{t+1} &= \min_{x \in [\ell_t, u_t]} p_t(x) \\ u_{t+1} &= \max_{x \in [\ell_t, u_t]} p_t(x) \end{aligned} \quad (9)$$

The resulting composition $p^* := p_T \circ p_{T-1} \circ \dots \circ p_1$ is optimal and the error is given by:

$$\max |1 - p^*(x)| = \min \max |1 - p(x)| = 1 - \ell_{T+1}. \quad (10)$$

Here I asked to explain the graph (I took a screenshot of the graph)

5

They are designing an *iterative polynomial approximation* to the matrix sign function. Think of it as:

- At iteration t you apply a degree- d polynomial p_t to your current value.
- After T iterations, you effectively apply the composition

$$p_T \circ p_{T-1} \circ \dots \circ p_1,$$

which should approximate $\text{sign}(x)$ well on $[\ell, 1]$.

Greedy choice of each polynomial

Key idea: once $p_1, \dots, p_{t-1}$ are fixed, the *image* of the original interval $[\ell, u]$ after these steps is another interval:

$$[\ell_t, u_t] := [p_{t-1} \circ \dots \circ p_1(\ell), p_{t-1} \circ \dots \circ p_1(u)].$$

Now at iteration t they choose p_t **greedily**:

- Given the current interval $[\ell_t, u_t]$, pick a degree- d polynomial p_t that makes the values $p_t(x)$ for $x \in [\ell_t, u_t]$ as close to 1 as possible:

$$p_t = \operatorname{argmin}_{p \in \mathcal{P}_d} \max_{x \in [\ell_t, u_t]} |1 - p(x)|.$$

- After applying p_t , the new outputs lie in another interval

$$\ell_{t+1} = \min_{x \in [\ell_t, u_t]} p_t(x), \quad u_{t+1} = \max_{x \in [\ell_t, u_t]} p_t(x),$$

which is exactly equation (8). This becomes the input range for the next step.

So each step ignores the *global* T -step optimization and only optimizes the *current* step—hence “greedy.” The theorem they reference shows this greedy strategy is actually *optimal* for their objective.

Relation to Figure 2

- Left plot (“Composition of polynomials”): shows the final composite polynomial $p_T \circ \dots \circ p_1$ for various methods (Polar Express, You's, Newton-Schulz, Jordan). For Polar Express, repeated greedy compression of $[\ell_t, u_t]$ toward 1 makes the composite polynomial almost flat at 1 over $[\ell, 1]$.
- Right plot (“Error convergence”): shows how the worst-case error $\max_{x \in [\ell, 1]} |1 - p_T \circ \dots \circ p_1(x)|$ decreases with iterations T . Polar Express drops extremely fast because each greedy step maximally “squeezes” the interval toward 1.

In panel (b), they visualize the first three polynomials p_1, p_2, p_3 and the sequence of intervals:

$$\ell_{t+1} = p_t(\ell_t), \quad u_{t+1} = 2 - \ell_{t+1},$$

showing how $[\ell_t, u_t]$ rapidly contracts around 1, explaining the fast convergence.

It seems like the LLM went off track in the beginning, explaining the bottom part of the page. (the entire page is fed as context but the screenshot focused on then graphs).

Here we return to the actual figure, though it might seem like the LLM was providing background for the response to the figure. However, this isn't optimal since we only want a response focused on the figure. Nonetheless, the LLM correctly explains what is happening

It also seems like the LLM ended up responding to the bottom part of figure 2 even though I focused my screenshot on the top graphs